

**O Programa de Apoio ao Desenvolvimento Tecnológico da Indústria de
Semicondutores (PADIS)**

Contador síncrono de 4 bits
Nome: Gabriel Vieira da Silva Leite

RELATÓRIO – UNIDADE 6

22 / 07 / 2026

1 – Introdução

Este trabalho teve como objetivo desenvolver um contador síncrono de 4 bits utilizando a linguagem SystemVerilog, além de elaborar um testbench capaz de verificar todas as funcionalidades especificadas. A atividade teve como finalidade reforçar os conceitos de projeto digital síncrono, utilização de clock, reset assíncrono, enable síncrono e técnicas de verificação funcional por meio de simulação.

O desenvolvimento foi realizado na plataforma EDA Playground utilizando um simulador compatível com SystemVerilog e a ferramenta EPWave para análise das formas de onda.

2 - Objetivos

Os principais objetivos deste trabalho foram:

- Desenvolver um contador síncrono crescente de 4 bits;
- Implementar reset assíncrono ativo em nível lógico baixo;
- Implementar enable síncrono ativo em nível lógico alto;
- Verificar o comportamento de overflow do contador;
- Desenvolver um testbench para validar todas as funcionalidades do circuito.

3 - Desenvolvimento do Contador

O módulo desenvolvido possui três entradas e uma saída:

- clk: sinal de clock responsável por sincronizar o funcionamento do contador;
- rst_n: reset assíncrono ativo em nível baixo;
- en: sinal de habilitação da contagem;
- count[3:0]: saída correspondente ao valor atual do contador.

O funcionamento do circuito foi implementado utilizando um bloco always_ff, sensível à borda de subida do clock (posedge clk) e à borda de descida do sinal de reset (negedge rst_n).

Inicialmente é verificado o estado do reset. Quando rst_n assume nível lógico baixo, o contador é imediatamente zerado, independentemente do clock ou do sinal de habilitação.

Caso o reset esteja inativo, a contagem somente ocorre quando o sinal en estiver em nível lógico alto. A cada borda positiva do clock, o valor armazenado é incrementado em uma unidade.

Como o contador possui apenas quatro bits, após atingir o valor decimal 15 (1111_2), o próximo incremento produz automaticamente o valor 0 (0000_2), caracterizando o overflow esperado.

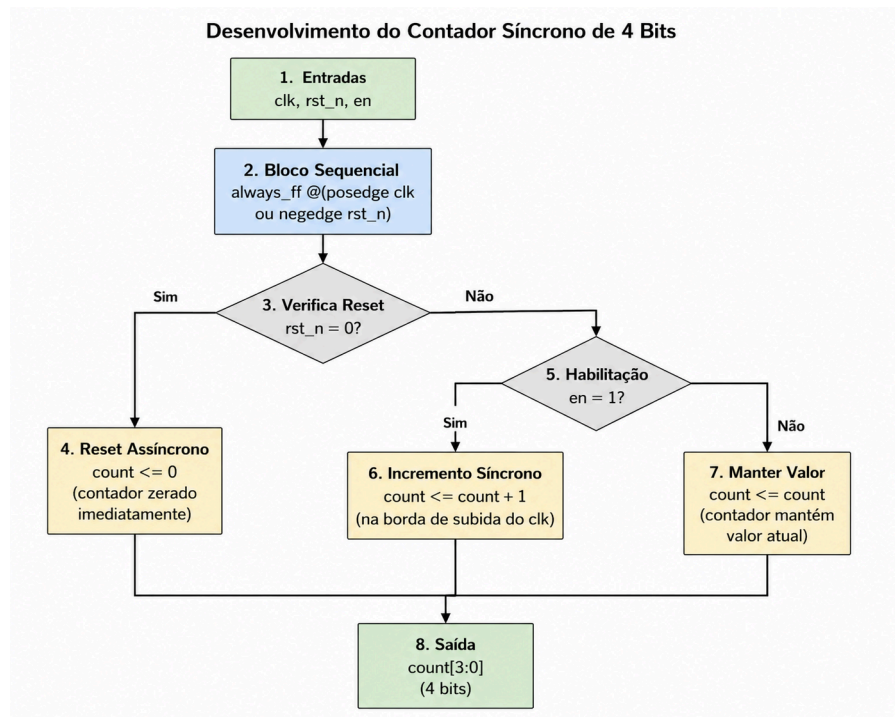


Figura 01: Desenvolvimento do Contador Síncrono de 4 Bits

4 - Resultados Experimentais

Foi desenvolvido um testbench responsável por estimular o circuito e verificar automaticamente todas as funcionalidades especificadas.

Inicialmente é gerado um clock periódico utilizando um bloco inicial contendo um comando `forever`, produzindo um sinal contínuo durante toda a simulação.

Em seguida foi implementada uma sequência de testes composta pelos seguintes cenários: aplicação do reset assíncrono;

- verificação da permanência do contador em zero durante o reset;
- habilitação da contagem ($en = 1$);
- contagem crescente de 0 até 15;
- verificação do overflow para zero;
- desabilitação do contador ($en = 0$);
- reabilitação da contagem;
- aplicação do reset durante a contagem.

Além dos estímulos, foram utilizados comandos `$display`, `$monitor` e `$error`, permitindo acompanhar a evolução dos sinais durante toda a simulação e identificar automaticamente qualquer comportamento incorreto. Também foi gerado o arquivo de formas de onda (`dump.vcd`) para visualização no EPWave.

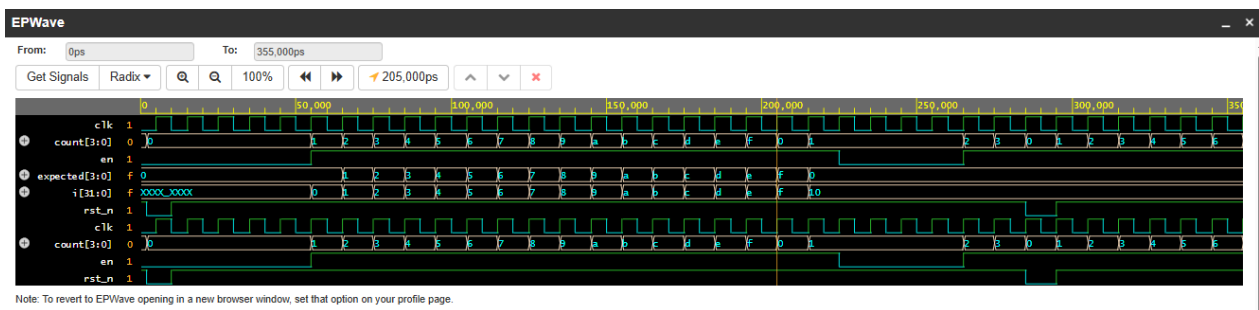


Figura 02: Forma de onda obtida na simulação do contador síncrono de 4 bits

5 - Conclusões

Durante a simulação observou-se que o contador apresentou o comportamento esperado em todos os cenários avaliados.

No teste de reset assíncrono, o contador foi imediatamente zerado assim que rst_n assume nível lógico baixo, independentemente da posição do clock.

Teste Reset Assíncrono: PASSOU

5000	1	0	0	0
10000	0	1	0	0
15000	1	1	0	0
20000	0	1	0	0
25000	1	1	0	0
30000	0	1	0	0
35000	1	1	0	0
40000	0	1	0	0
45000	1	1	0	0
50000	0	1	0	0

Figura 03: Teste do Reset Assíncrono. Respetivas Colunas: Tempo; Clock; Reset; Enable; Contador.

Quando o sinal permanece desabilitado, o valor armazenado no contador permaneceu constante durante vários ciclos de clock.

Teste Overflow: PASSOU

225000	1	1	0	1
230000	0	1	0	1
235000	1	1	0	1
240000	0	1	0	1
245000	1	1	0	1
250000	0	1	0	1
255000	1	1	0	1
260000	0	1	0	1

Figura 04: Teste do Enable. Respetivas Colunas: Tempo; Clock; Reset; Enable; Contador.

Após a ativação do enable, verificou-se a sequência correta de contagem crescente até o valor decimal 15. No ciclo seguinte ocorreu automaticamente o retorno para zero, comprovando o correto funcionamento do overflow.

```

Teste Enable=0: PASSOU
55000 1 1 1 1
60000 0 1 1 1
65000 1 1 1 2
70000 0 1 1 2
75000 1 1 1 3
80000 0 1 1 3
85000 1 1 1 4
90000 0 1 1 4
95000 1 1 1 5
100000 0 1 1 5
105000 1 1 1 6
110000 0 1 1 6
115000 1 1 1 7
120000 0 1 1 7
125000 1 1 1 8
130000 0 1 1 8
135000 1 1 1 9
140000 0 1 1 9
145000 1 1 1 10
150000 0 1 1 10
155000 1 1 1 11
160000 0 1 1 11
165000 1 1 1 12
170000 0 1 1 12
175000 1 1 1 13
180000 0 1 1 13
185000 1 1 1 14
190000 0 1 1 14
195000 1 1 1 15
200000 0 1 1 15
205000 1 1 1 0
210000 0 1 1 0
Teste Contagem Crescente: PASSOU
215000 1 1 1 1
220000 0 1 1 1

```

Figura 05: Teste da Contagem. Respectivas Colunas: Tempo; Clock; Reset; Enable; Contador.

Também foram realizados testes alternando o sinal de enable durante a execução e aplicando o reset no meio da contagem. Em ambos os casos o circuito respondeu conforme esperado, demonstrando o correto funcionamento da lógica implementada.

```

Teste Desabilitação: PASSOU
265000 1 1 1 2
270000 0 1 1 2
Teste Reabilitação: PASSOU
275000 1 1 1 3
280000 0 1 1 3
285000 1 0 1 0
Teste Reset Durante Contagem: PASSOU
290000 0 0 1 0
295000 1 1 1 1
300000 0 1 1 1
305000 1 1 1 2
310000 0 1 1 2
315000 1 1 1 3
320000 0 1 1 3
325000 1 1 1 4
330000 0 1 1 4
Todos os testes foram executados.
335000 1 1 1 5
340000 0 1 1 5
345000 1 1 1 6
350000 0 1 1 6

```

Figura 06: Teste de alternância dos sinais. Respectivas Colunas: Tempo; Clock; Reset; Enable; Contador.

A análise das formas de onda no EPWave, mostrado na figura 02, confirmou visualmente todos esses comportamentos.

6 - Conclusão

O contador síncrono de 4 bits foi desenvolvido com sucesso utilizando SystemVerilog, atendendo integralmente aos requisitos propostos.

O uso do bloco `always_ff` e de atribuições nonblocking (`<=`) proporcionou uma implementação compatível com as boas práticas de descrição de lógica sequencial.

O testbench desenvolvido permitiu validar automaticamente todas as funcionalidades do circuito, incluindo reset assíncrono, enable síncrono, contagem crescente, overflow e cenários de transição.

A utilização do EDA Playground juntamente com o EPWave facilitou a verificação funcional do projeto, permitindo observar tanto as mensagens de simulação quanto as formas de onda geradas, contribuindo para a compreensão do funcionamento do circuito e do processo de verificação em projetos digitais.

7 - Referências Bibliográficas

NAVABI, Zainalabedin. **Verilog Digital System Design: RTL Design, Synthesis and Verification**. 2. ed. New York: McGraw-Hill, 2006.

EDA PLAYGROUND. *EDA Playground: Online Verilog, SystemVerilog and VHDL Simulator*. Disponível em: <https://www.edaplayground.com>. Acesso em: 22 jul. 2026.